Breast Cancer Prediction Project

In [1]:
```
#import libraries
import numpy as np
import pandas as pd
import os
import matplotlib.pyplot as plt
from sklearn.metrics import classification_report
from sklearn.metrics import confusion_matrix
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split
from sklearn.model_selection import cross_val_score
from sklearn.model_selection import KFold
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import GridSearchCV
from sklearn.svm import SVC
import time
from scipy.stats import norm
import seaborn as sns # data visualization
```

In [2]:
```
data = pd.read_csv('data.csv', index_col=False)
data.head(5)
```

Out[2]:

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	sr
0	842302	M	17.99	10.38	122.80	1001.0	
1	842517	M	20.57	17.77	132.90	1326.0	
2	84300903	M	19.69	21.25	130.00	1203.0	
3	84348301	M	11.42	20.38	77.58	386.1	
4	84358402	M	20.29	14.34	135.10	1297.0	

5 rows × 32 columns

In [3]:
```
print(data.shape)
```

```
(569, 32)
```

In [4]:
```
data.describe()
```

Out[4]:

	id	radius_mean	texture_mean	perimeter_mean	area_mean	smo
count	5.690000e+02	569.000000	569.000000	569.000000	569.000000	
mean	3.037183e+07	14.127292	19.289649	91.969033	654.889104	
std	1.250206e+08	3.524049	4.301036	24.298981	351.914129	
min	8.670000e+03	6.981000	9.710000	43.790000	143.500000	
25%	8.692180e+05	11.700000	16.170000	75.170000	420.300000	
50%	9.060240e+05	13.370000	18.840000	86.240000	551.100000	
75%	8.813129e+06	15.780000	21.800000	104.100000	782.700000	
max	9.113205e+08	28.110000	39.280000	188.500000	2501.000000	

8 rows × 31 columns

In [5]:
```
#Data visualisation and pre-processing
data['diagnosis'] = data['diagnosis'].apply(lambda x: '1' if x == 'M' else '
data = data.set_index('id')
```

In [6]:
```
df = pd.read_csv('data.csv', index_col=False)
```

In [7]:
```
df.diagnosis.unique()
```

Out[7]:
```
array(['M', 'B'], dtype=object)
```

In [8]:
```
diag_gr = df.groupby('diagnosis', axis=0)
pd.DataFrame(diag_gr.size(), columns=['# of observations'])
```

```
/var/folders/g7/8fm0cc6d7p13pn53xrhdqw5c0000gn/T/ipykernel_76386/562642983.p
y:1: FutureWarning: The 'axis' keyword in DataFrame.groupby is deprecated an
d will be removed in a future version.
  diag_gr = df.groupby('diagnosis', axis=0)
```

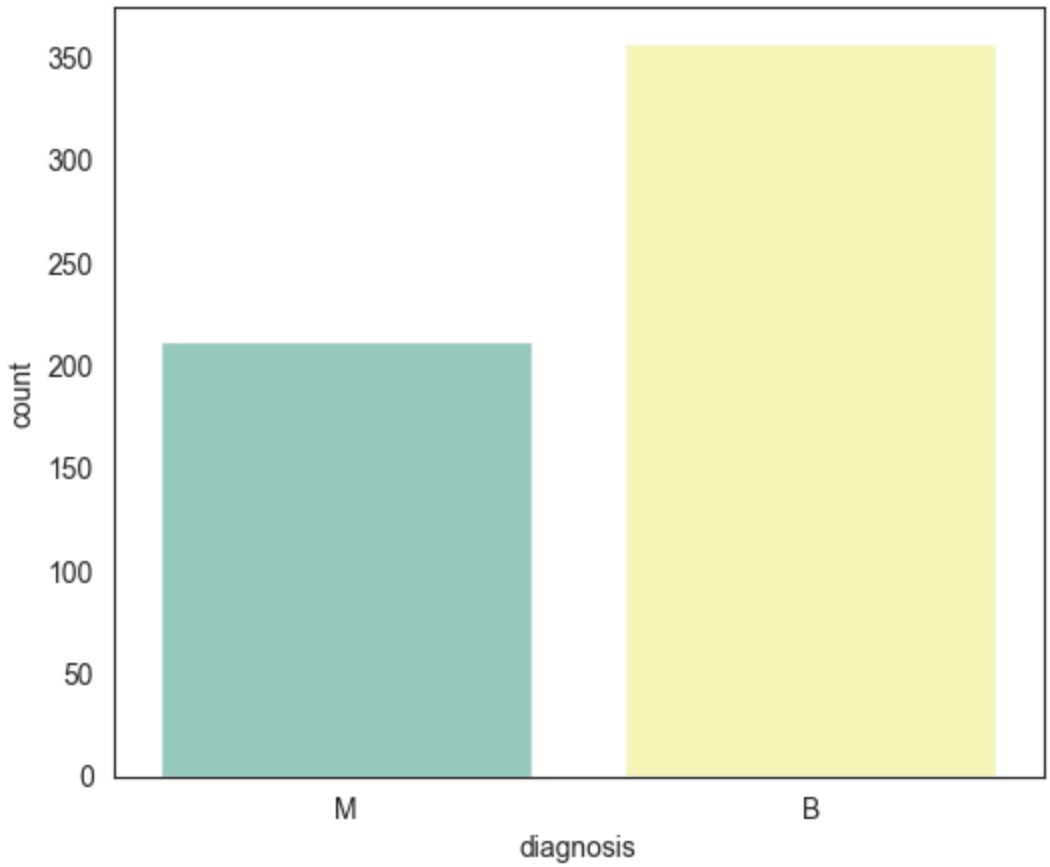
Out[8]:

diagnosis	# of observations
B	357
M	212

In [9]:
```
sns.set_style("white")
sns.set_context({"figure.figsize": (6, 5)})
sns.countplot(x=df['diagnosis'],label='Count',palette="Set3")
# plt.savefig('output1.png')
```

```
/var/folders/g7/8fm0cc6d7p13pn53xrhdqw5c0000gn/T/ipykernel_76386/733088857.p
y:3: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed
in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the
same effect.

  sns.countplot(x=df['diagnosis'],label='Count',palette="Set3")
```

Out[9]: <Axes: xlabel='diagnosis', ylabel='count'>



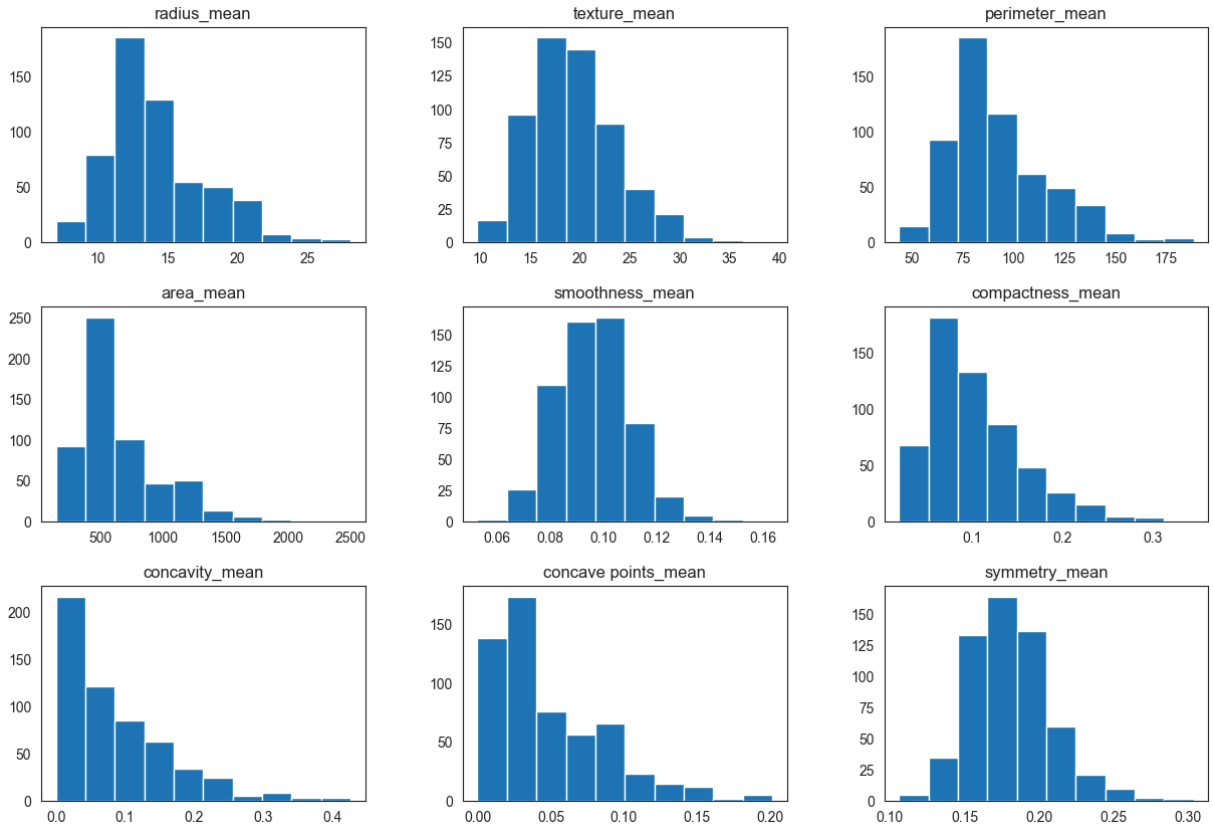
In [10]:
```
print(data.groupby('diagnosis').size())
```

```
diagnosis
0    357
1    212
dtype: int64
```

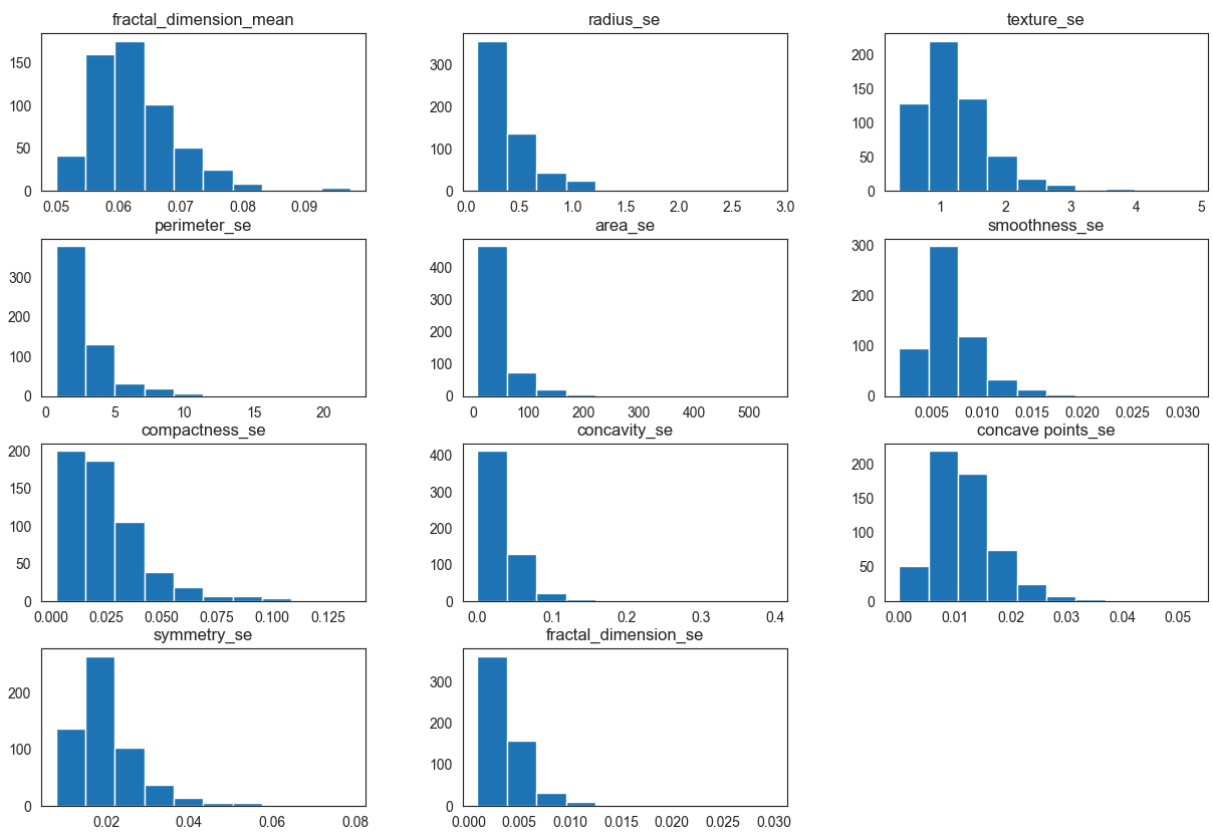
In [11]:
```
df_id_diag=df.loc[:,["id","diagnosis"]]
df_diag=df.loc[:,["diagnosis"]]
df_mean=df.iloc[:,1:11]
df_se=df.iloc[:,11:22]
df_worst=df.iloc[:,23:]
print(df_id_diag.columns)
```

```
Index(['id', 'diagnosis'], dtype='object')
```

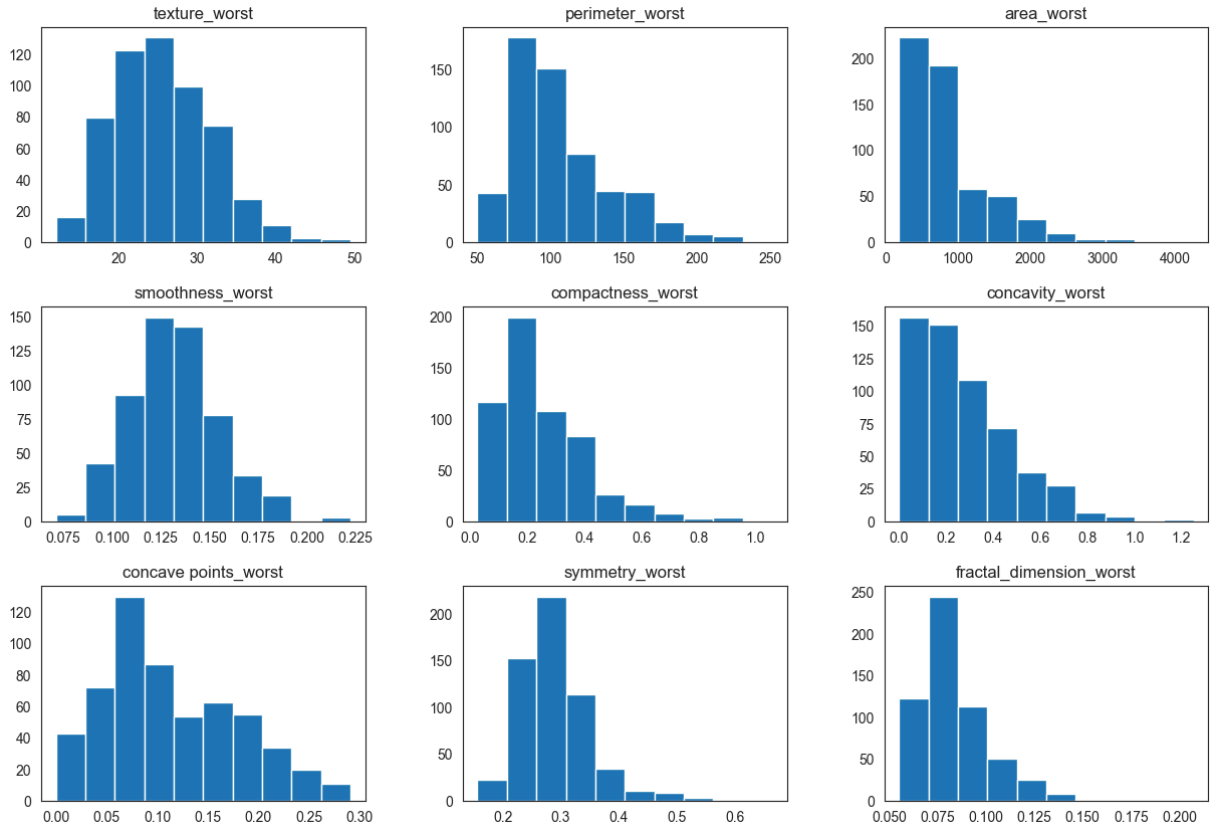
In [12]:
```
hist_mean=df_mean.hist(bins=10, figsize=(15, 10),grid=False,)
```



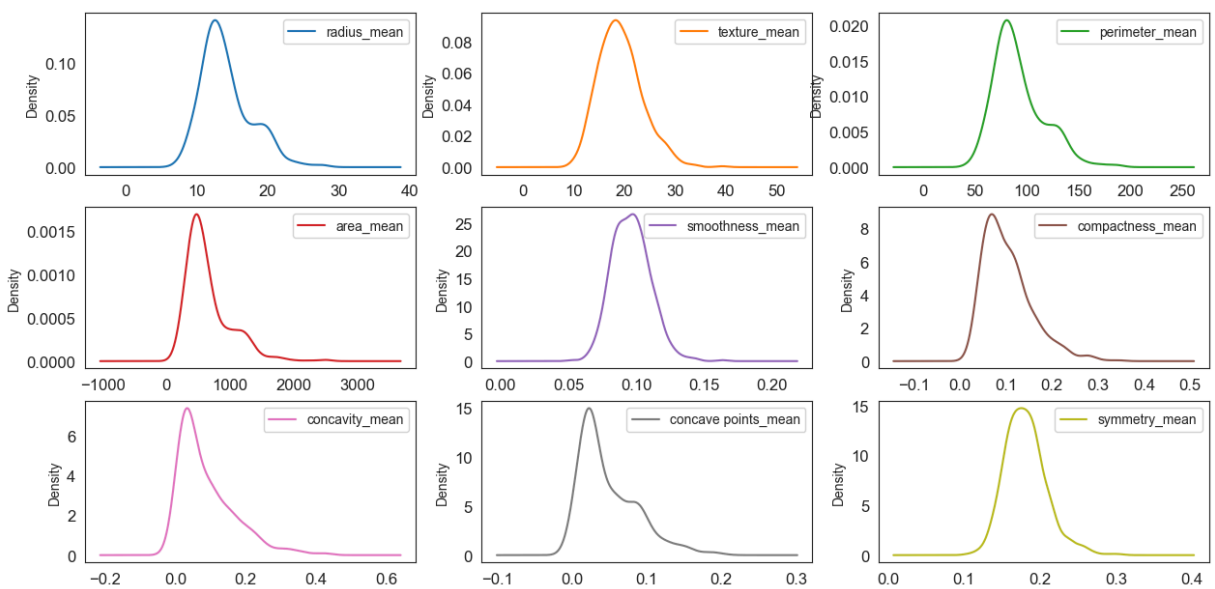
In [13]:
```
hist_se=df_se.hist(bins=10, figsize=(15, 10),grid=False,)
```



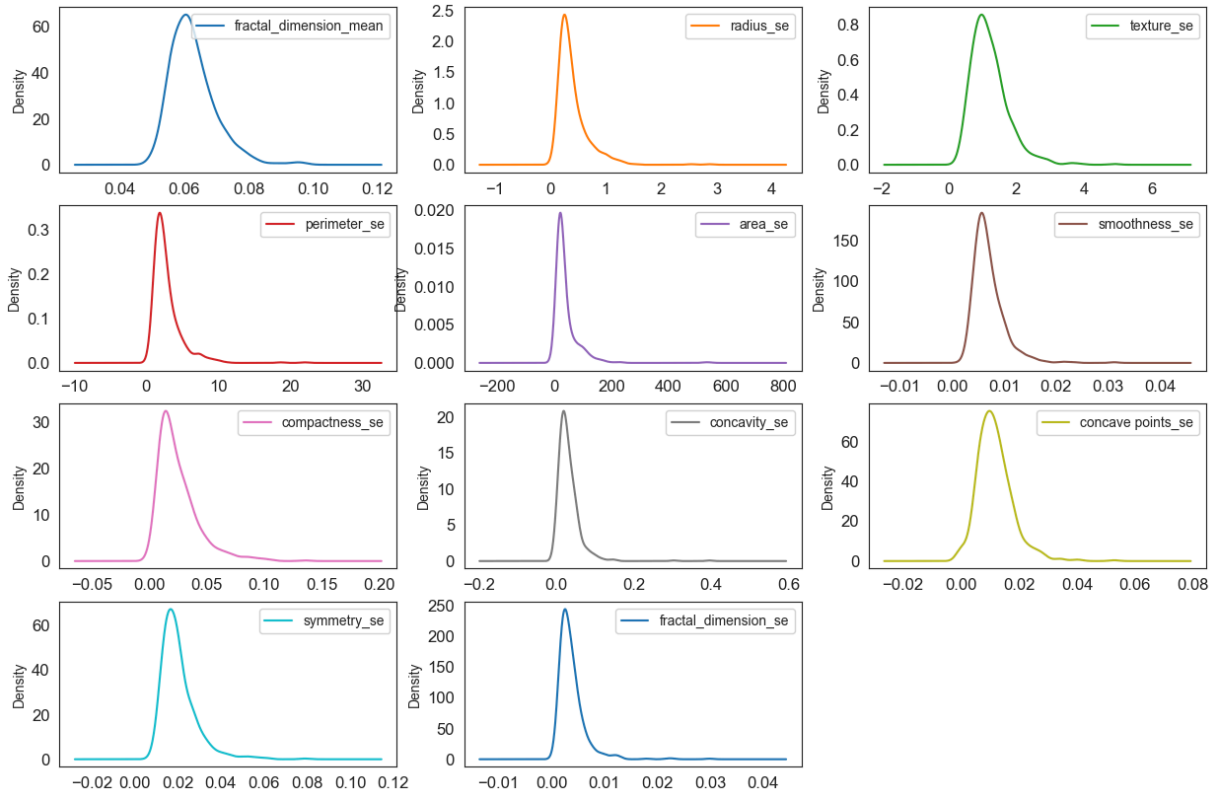
In [14]:
```
hist_worst=df_worst.hist(bins=10, figsize=(15, 10),grid=False,)
```



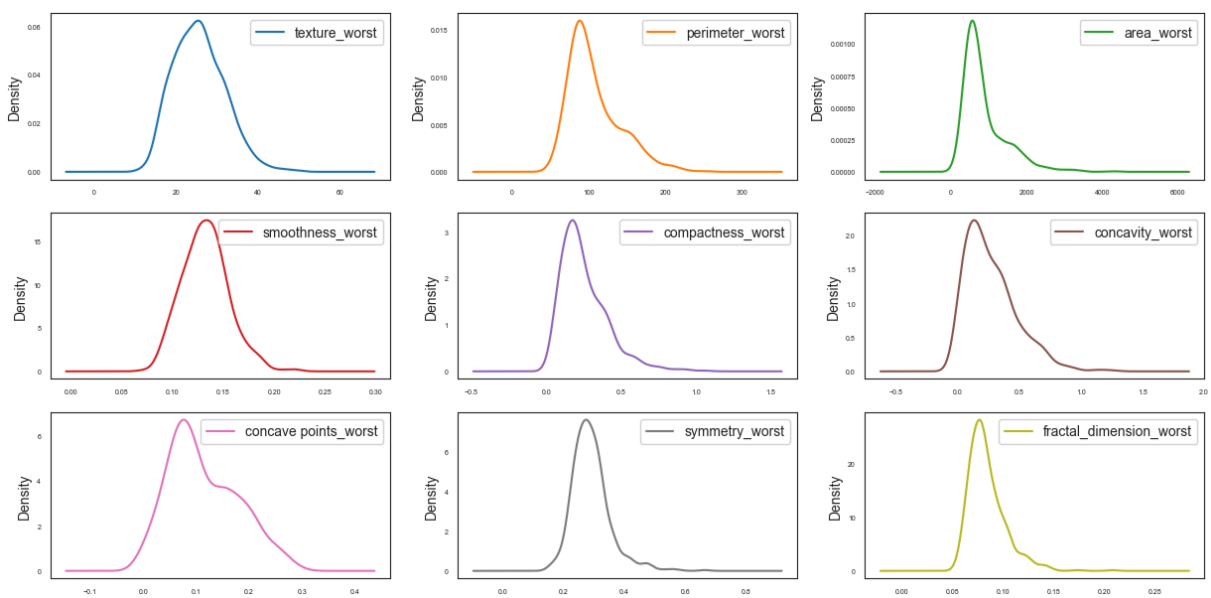
In [15]:
```
plt = df_mean.plot(kind= 'density', subplots=True, layout=(4,3), sharex=Fals
                   sharey=False, fontsize=12, figsize=(15,10))
```



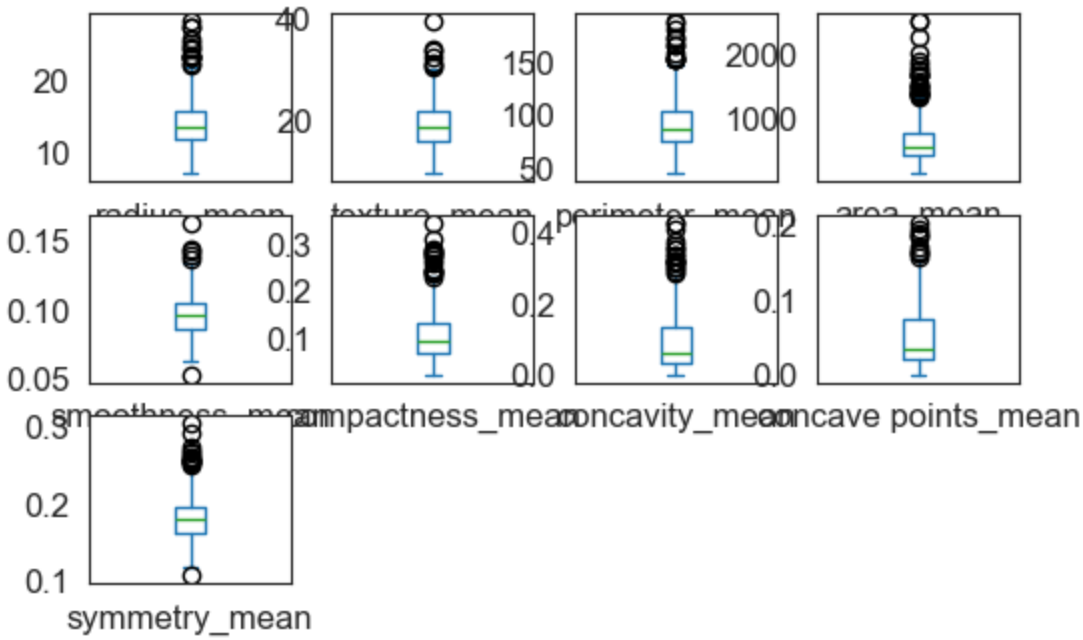
In [16]:
```
#Density Plots
plt = df_se.plot(kind= 'density', subplots=True, layout=(4,3), sharex=False,
                   sharey=False, fontsize=12, figsize=(15,10))
```



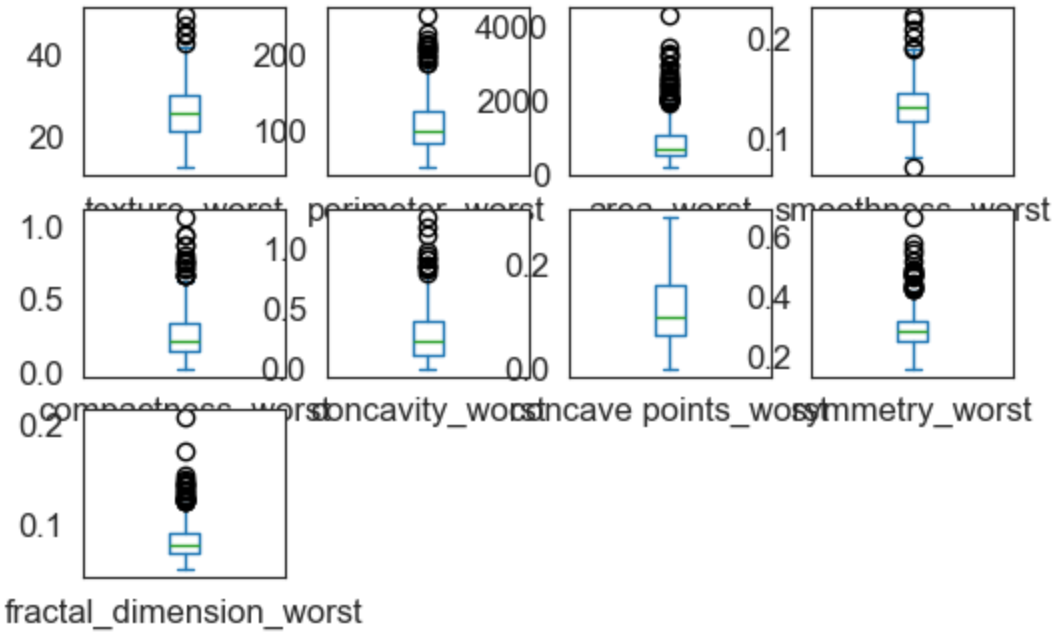
In [17]:
```
#Density Plots
plt = df_worst.plot(kind= 'kde', subplots=True, layout=(4,3), sharex=False,
```



In [18]:
```
# box and whisker plots
plt=df_mean.plot(kind= 'box' , subplots=True, layout=(4,4), sharex=False, sh
                 fontsize=12)
```

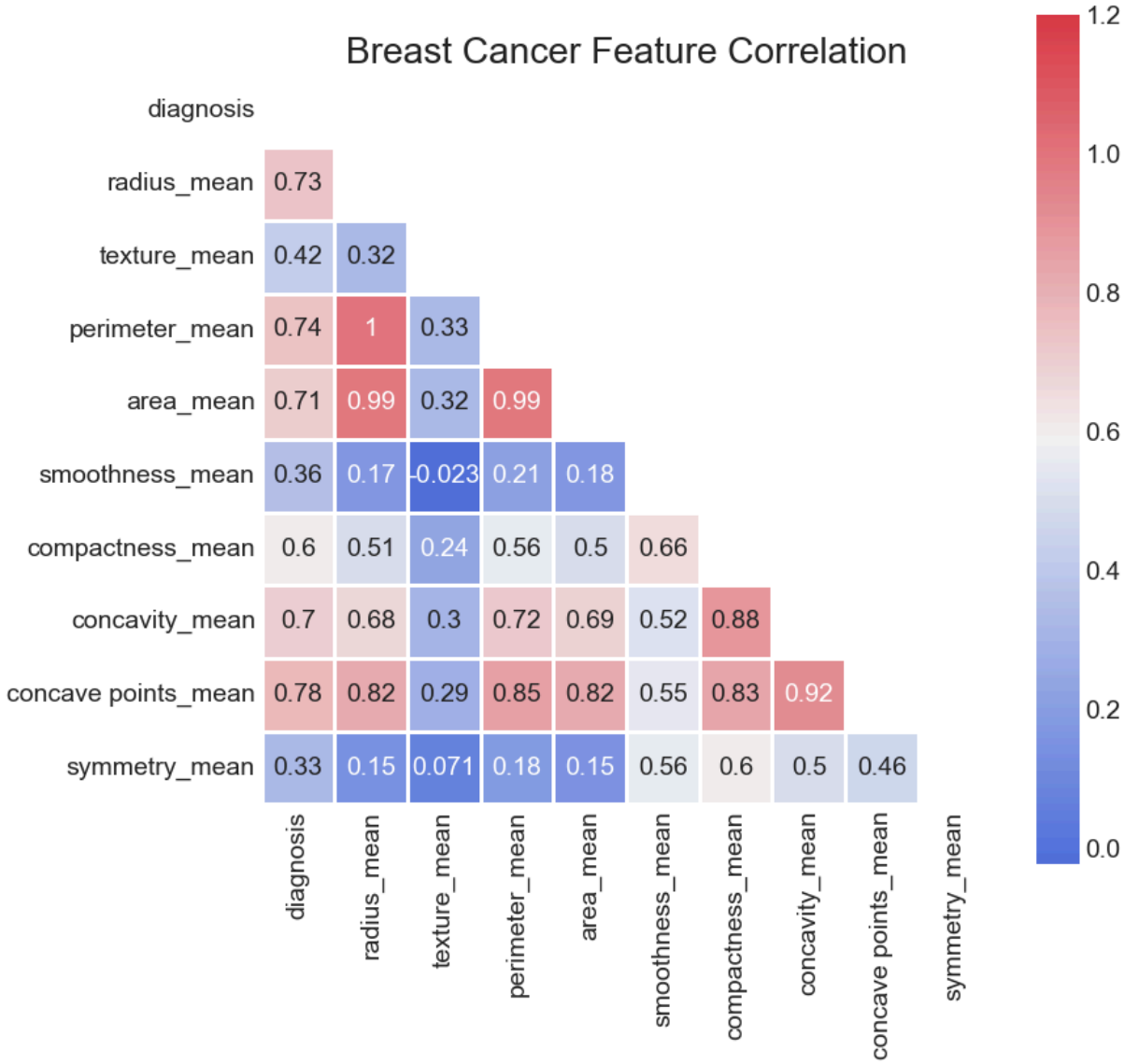


In [19]:
```
plt=df_worst.plot(kind= 'box' , subplots=True, layout=(4,4), sharex=False, s
                  fontsize=12)
```



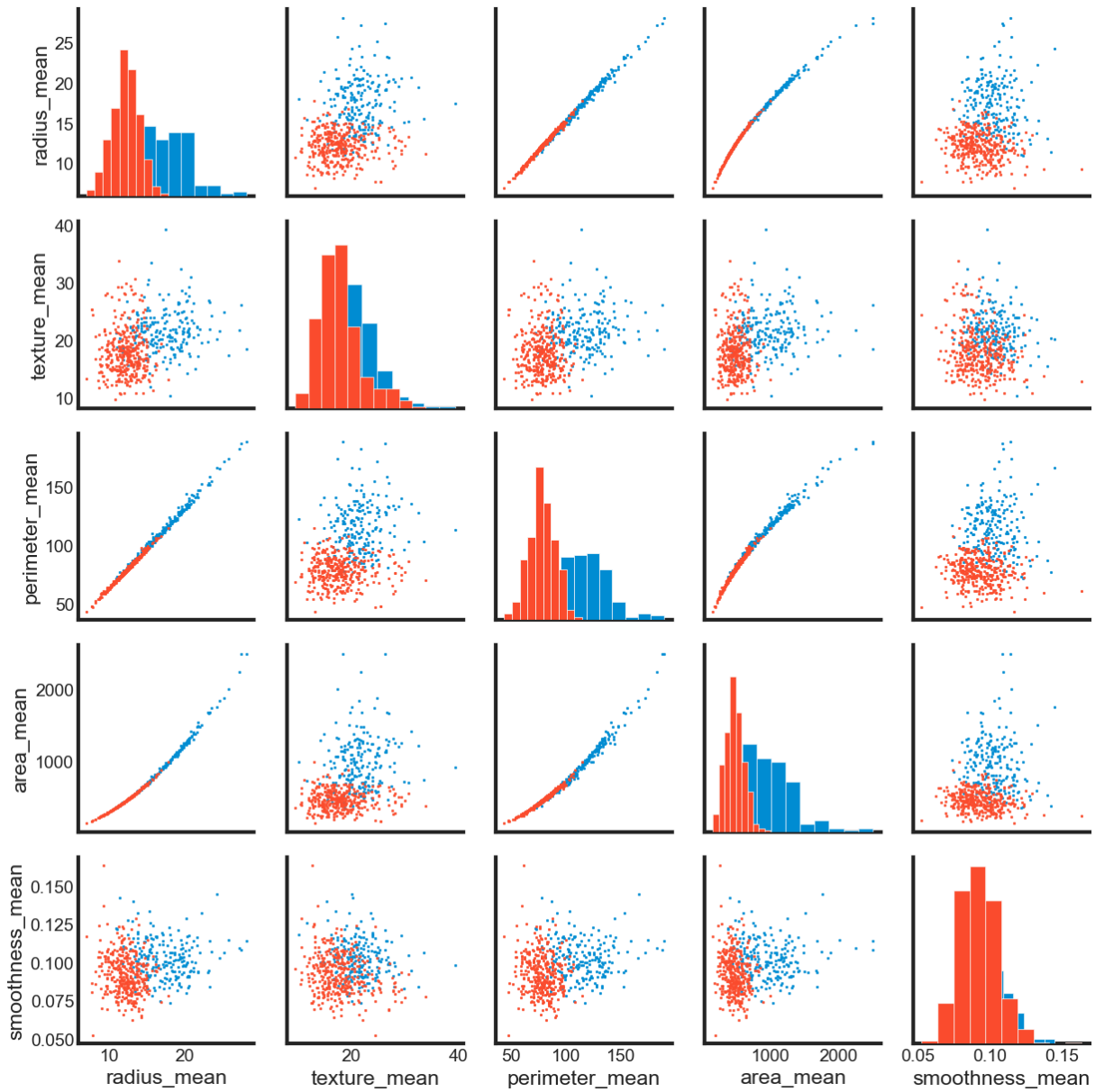
In [20]:
```
import matplotlib.pyplot as plt
plt.style.use('fivethirtyeight')
sns.set_style("white")
df_mean['diagnosis'] = df_mean['diagnosis'].apply(lambda x: '1' if x == 'M'
corr = df_mean.corr()
mask = np.zeros_like(corr)
mask[np.triu_indices_from(mask)] = True
df1, ax = plt.subplots(figsize=(8, 8))
plt.title('Breast Cancer Feature Correlation')
cmap = sns.diverging_palette(260, 10, as_cmap=True)
sns.heatmap(corr, vmax=1.2, square='square', cmap=cmap, mask=mask,
            ax=ax,annot=True, fmt='.2g',linewidths=2)
```

Out[20]: <Axes: title={'center': 'Breast Cancer Feature Correlation'}>



In [21]:

```
plt.style.use('fivethirtyeight')
sns.set_style("white")
# df = pd.read_csv('data_clean.csv', index_col=False)
g = sns.PairGrid(df[[df.columns[1],df.columns[2], df.columns[3],
                     df.columns[4], df.columns[5], df.columns[6]]], hue='dia
g = g.map_diag(plt.hist)
g = g.map_offdiag(plt.scatter, s = 3)
```



In [22]:
```
#Split data
Y = data['diagnosis'].values
X = data.drop('diagnosis', axis=1).values
X_train, X_test, Y_train, Y_test = train_test_split (X, Y, test_size = 0.20,
```

In [23]:
```
#Using diffrenet Algorithms
models_list = []
models_list.append(('CART', DecisionTreeClassifier()))
models_list.append(('SVM', SVC()))
models_list.append(('NB', GaussianNB()))
models_list.append(('KNN', KNeighborsClassifier()))
```

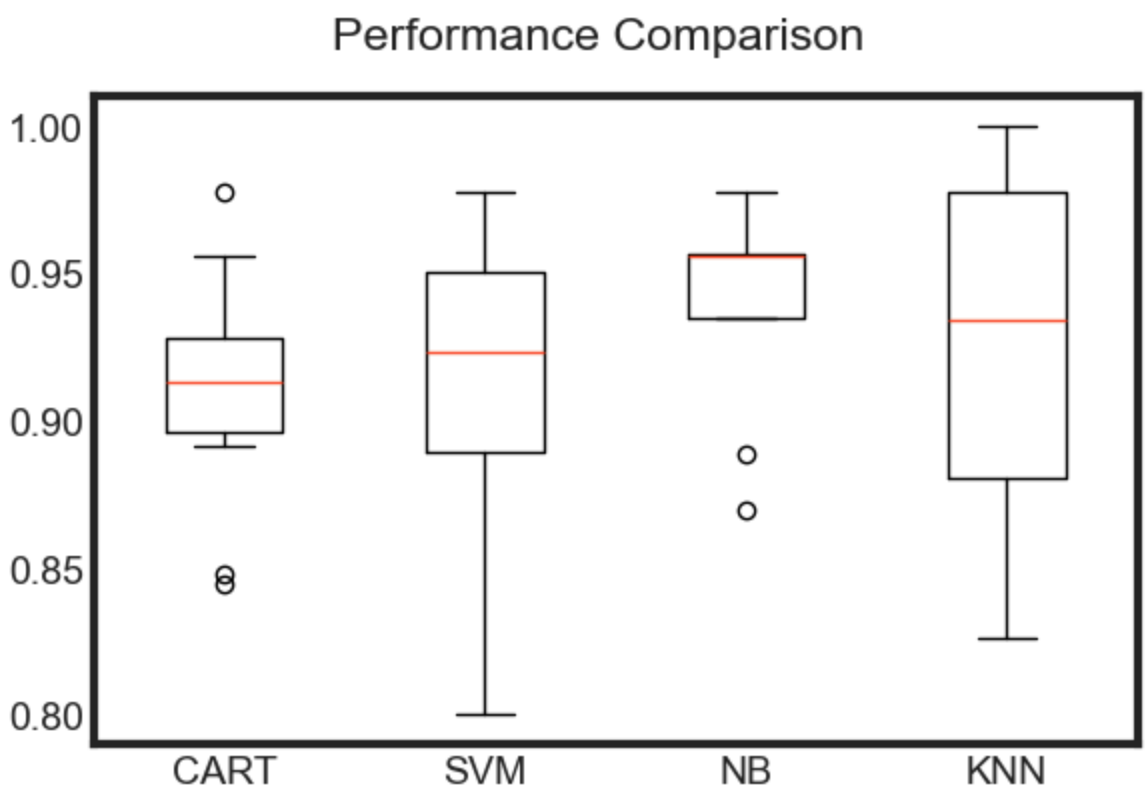
In [24]:
```
num_folds = 10
results = []
names = []
for name, model in models_list:
    kfold = KFold(n_splits=num_folds)#, random_state=123)
    start = time.time()
    cv_results = cross_val_score(model, X_train, Y_train, cv=kfold, scoring=
```

```
    end = time.time()
    results.append(cv_results)
    names.append(name)
    print( "%s: %f (%f) (run time: %f)" % (name, cv_results.mean(), cv_resul
```

```
CART: 0.910048 (0.039693) (run time: 0.064221)
SVM: 0.907681 (0.054723) (run time: 0.016534)
NB: 0.940773 (0.033921) (run time: 0.006639)
KNN: 0.927729 (0.055250) (run time: 0.306950)
```

In [25]:

```
fig = plt.figure(figsize=(6,4))
fig.suptitle('Performance Comparison')
ax = fig.add_subplot(111)
plt.boxplot(results)
ax.set_xticklabels(names)
plt.show()
```



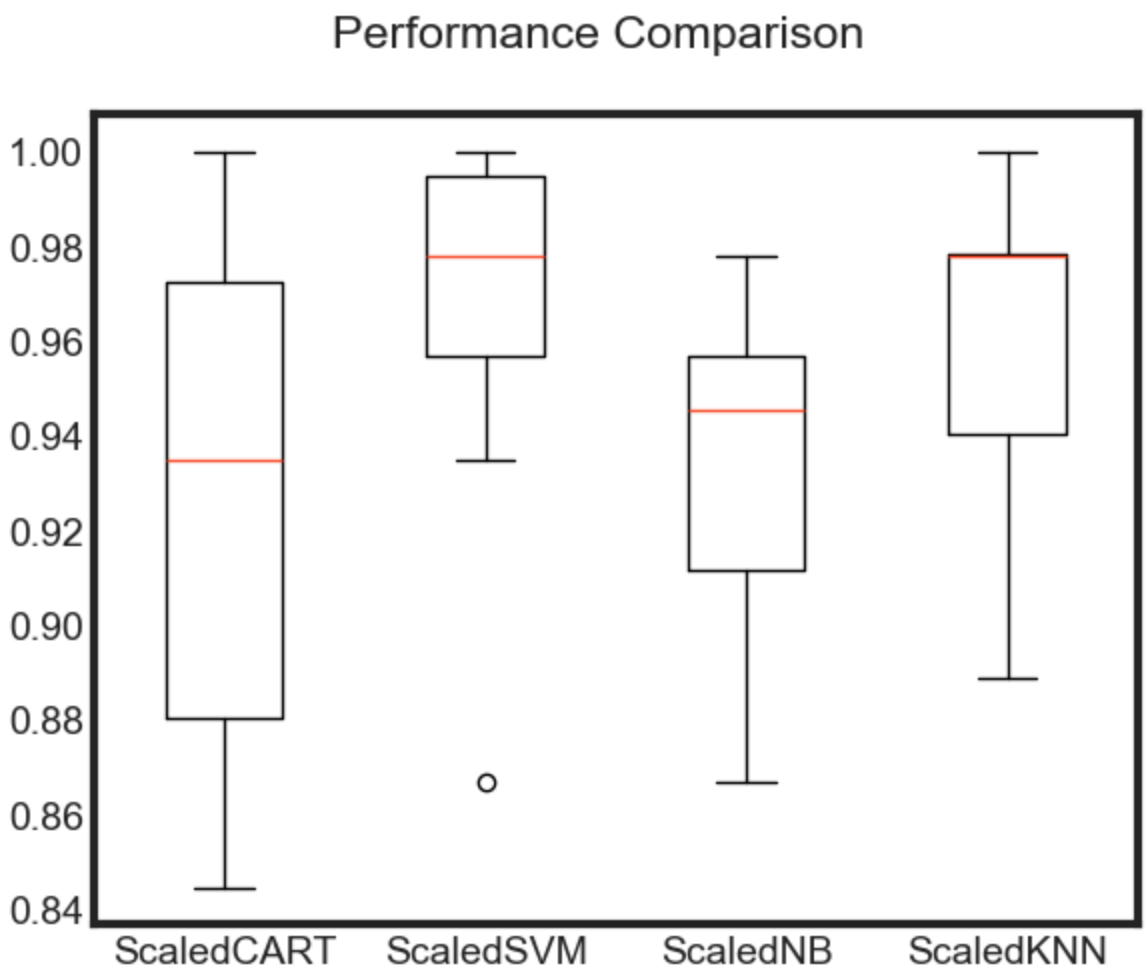
In [26]:

```
import warnings
pipelines = []
pipelines.append(('ScaledCART', Pipeline([('Scaler', StandardScaler()),('CAR
pipelines.append(('ScaledSVM', Pipeline([('Scaler', StandardScaler()),('SVM'
pipelines.append(('ScaledNB', Pipeline([('Scaler', StandardScaler()),('NB',G
pipelines.append(('ScaledKNN', Pipeline([('Scaler', StandardScaler()),('KNN'
results = []
names = []
with warnings.catch_warnings():
    warnings.simplefilter("ignore")
    kfold = KFold(n_splits=num_folds)#, random_state=123)
    for name, model in pipelines:
        start = time.time()
        cv_results = cross_val_score(model, X_train, Y_train, cv=kfold, scor
        end = time.time()
```

```
        results.append(cv_results)
        names.append(name)
        print( "%s: %f (%f) (run time: %f)" % (name, cv_results.mean(), cv_r
```

```
ScaledCART: 0.927729 (0.050025) (run time: 0.065244)
ScaledSVM: 0.964879 (0.038621) (run time: 0.017424)
ScaledNB: 0.931932 (0.038625) (run time: 0.009207)
ScaledKNN: 0.958357 (0.038595) (run time: 0.014492)
```

In [27]:
```
fig = plt.figure(figsize=(6,5))
fig.suptitle('Performance Comparison')
ax = fig.add_subplot(111)
plt.boxplot(results)
ax.set_xticklabels(names)
plt.show()
```



In [28]:
```
scaler = StandardScaler().fit(X_train)
rescaledX = scaler.transform(X_train)
c_values = [0.1, 0.3, 0.5, 0.7, 0.9, 1.0, 1.3, 1.5, 1.7, 2.0]
kernel_values = ['linear', 'poly', 'rbf', 'sigmoid']
param_grid = dict(C=c_values, kernel=kernel_values)
model = SVC()
kfold = KFold(n_splits=num_folds)#, random_state=21)
grid = GridSearchCV(estimator=model, param_grid=param_grid, scoring='accurac
grid_result = grid.fit(rescaledX, Y_train)
print("Best: %f using %s" % (grid_result.best_score_, grid_result.best_param
```

```
means = grid_result.cv_results_['mean_test_score']
stds = grid_result.cv_results_['std_test_score']
params = grid_result.cv_results_['params']
for mean, stdev, param in zip(means, stds, params):
    print("%f (%f) with: %r" % (mean, stdev, param))
```

```
Best: 0.969324 using {'C': 2.0, 'kernel': 'rbf'}
0.964976 (0.026211) with: {'C': 0.1, 'kernel': 'linear'}
0.828551 (0.054707) with: {'C': 0.1, 'kernel': 'poly'}
0.940725 (0.038380) with: {'C': 0.1, 'kernel': 'rbf'}
0.949469 (0.032899) with: {'C': 0.1, 'kernel': 'sigmoid'}
0.962754 (0.029531) with: {'C': 0.3, 'kernel': 'linear'}
0.863720 (0.050997) with: {'C': 0.3, 'kernel': 'poly'}
0.956039 (0.032900) with: {'C': 0.3, 'kernel': 'rbf'}
0.960386 (0.029499) with: {'C': 0.3, 'kernel': 'sigmoid'}
0.956184 (0.030988) with: {'C': 0.5, 'kernel': 'linear'}
0.879034 (0.053507) with: {'C': 0.5, 'kernel': 'poly'}
0.964879 (0.030054) with: {'C': 0.5, 'kernel': 'rbf'}
0.956087 (0.027848) with: {'C': 0.5, 'kernel': 'sigmoid'}
0.954010 (0.031644) with: {'C': 0.7, 'kernel': 'linear'}
0.885604 (0.038275) with: {'C': 0.7, 'kernel': 'poly'}
0.967053 (0.037461) with: {'C': 0.7, 'kernel': 'rbf'}
0.949565 (0.027831) with: {'C': 0.7, 'kernel': 'sigmoid'}
0.951836 (0.028830) with: {'C': 0.9, 'kernel': 'linear'}
0.887826 (0.039039) with: {'C': 0.9, 'kernel': 'poly'}
0.967053 (0.037461) with: {'C': 0.9, 'kernel': 'rbf'}
0.947391 (0.032846) with: {'C': 0.9, 'kernel': 'sigmoid'}
0.954010 (0.026552) with: {'C': 1.0, 'kernel': 'linear'}
0.890048 (0.038398) with: {'C': 1.0, 'kernel': 'poly'}
0.967101 (0.033160) with: {'C': 1.0, 'kernel': 'rbf'}
0.947391 (0.032846) with: {'C': 1.0, 'kernel': 'sigmoid'}
0.956184 (0.025767) with: {'C': 1.3, 'kernel': 'linear'}
0.894396 (0.039509) with: {'C': 1.3, 'kernel': 'poly'}
0.967150 (0.028285) with: {'C': 1.3, 'kernel': 'rbf'}
0.947391 (0.029759) with: {'C': 1.3, 'kernel': 'sigmoid'}
0.958357 (0.024768) with: {'C': 1.5, 'kernel': 'linear'}
0.898792 (0.033329) with: {'C': 1.5, 'kernel': 'poly'}
0.967150 (0.028285) with: {'C': 1.5, 'kernel': 'rbf'}
0.940821 (0.039361) with: {'C': 1.5, 'kernel': 'sigmoid'}
0.956135 (0.021744) with: {'C': 1.7, 'kernel': 'linear'}
0.900966 (0.034861) with: {'C': 1.7, 'kernel': 'poly'}
0.967150 (0.024547) with: {'C': 1.7, 'kernel': 'rbf'}
0.940918 (0.037900) with: {'C': 1.7, 'kernel': 'sigmoid'}
0.956135 (0.021744) with: {'C': 2.0, 'kernel': 'linear'}
0.907536 (0.034327) with: {'C': 2.0, 'kernel': 'poly'}
0.969324 (0.022458) with: {'C': 2.0, 'kernel': 'rbf'}
0.932029 (0.028300) with: {'C': 2.0, 'kernel': 'sigmoid'}
```

In [29]:
```
print(means.max(),means.size)
means.argmax()
```

```
0.9693236714975846 40
```

Out[29]:
```
np.int64(38)
```

In [30]:
```
grid_result.best_params_
```

Out[30]: {'C': 2.0, 'kernel': 'rbf'}

In [31]:
```
grid_result.best_params_['kernel']
```

Out[31]: 'rbf'

In [32]:
```
# prepare the model
with warnings.catch_warnings():
    warnings.simplefilter("ignore")
    scaler = StandardScaler().fit(X_train)
X_train_scaled = scaler.transform(X_train)
model = SVC(C=2.0, kernel='rbf')
start = time.time()
model.fit(X_train_scaled, Y_train)
end = time.time()
print( "Run Time: %f" % (end-start))
```

```
Run Time: 0.006257
```

In [33]:
```
# estimate accuracy on test dataset
with warnings.catch_warnings():
    warnings.simplefilter("ignore")
    X_test_scaled = scaler.transform(X_test)
predictions = model.predict(X_test_scaled)
```

In [34]:
```
print("Accuracy score %f" % accuracy_score(Y_test, predictions))
print(classification_report(Y_test, predictions))
```

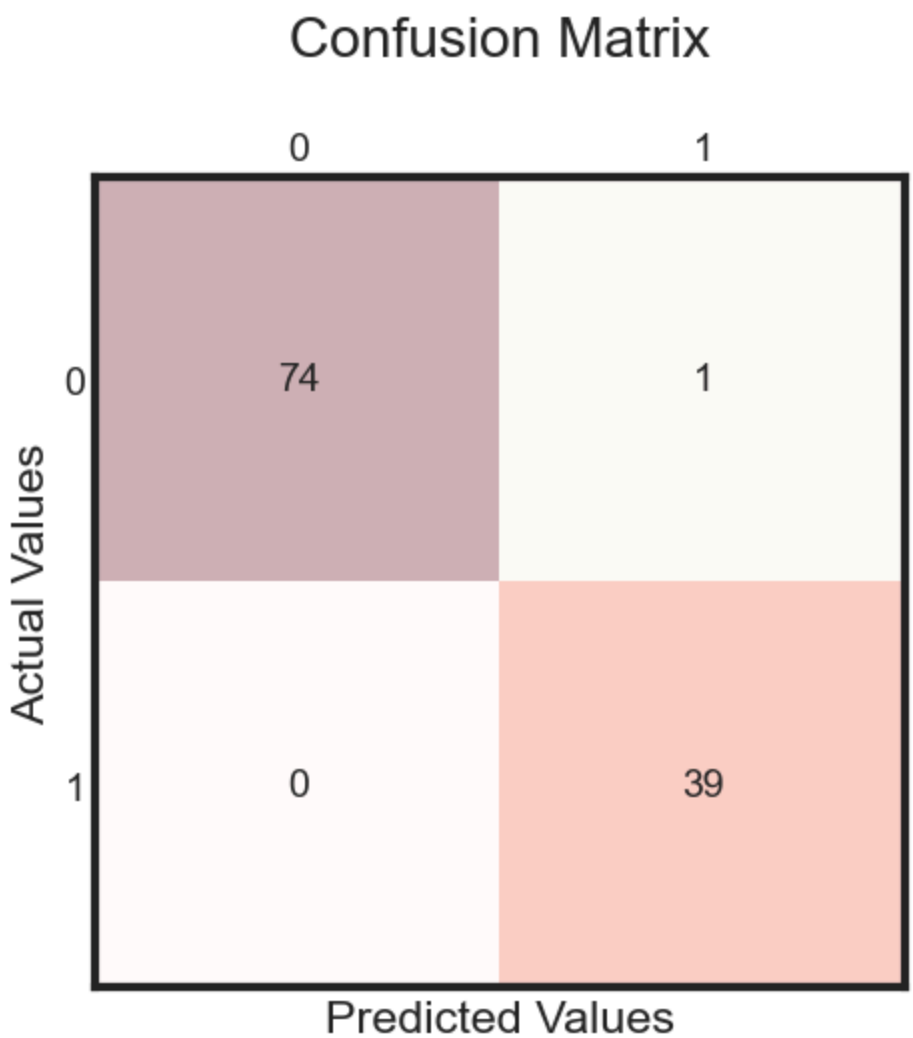
```
Accuracy score 0.991228
              precision    recall  f1-score   support

           0       1.00      0.99      0.99        75
           1       0.97      1.00      0.99        39

    accuracy                           0.99       114
   macro avg       0.99      0.99      0.99       114
weighted avg       0.99      0.99      0.99       114
```

In [35]:
```
print(confusion_matrix(Y_test, predictions))
```

```
[[74  1]
 [ 0 39]]
```

In [36]:
```
cm = confusion_matrix(Y_test, predictions)
%matplotlib inline
import matplotlib.pyplot as plt
from IPython.display import Image, display
fig, ax = plt.subplots(figsize=(5, 5))
ax.matshow(cm, cmap=plt.cm.Reds, alpha=0.3)
for i in range(cm.shape[0]):
     for j in range(cm.shape[1]):
         ax.text(x=j, y=i,
                s=cm[i, j],
                va='center', ha='center')
plt.xlabel('Predicted Values', )
```

```
plt.ylabel('Actual Values')
plt.title('Confusion Matrix\n')
plt.show()
print(classification_report(Y_test, predictions ))
```




```
              precision    recall  f1-score   support

           0       1.00      0.99      0.99        75
           1       0.97      1.00      0.99        39

    accuracy                           0.99       114
   macro avg       0.99      0.99      0.99       114
weighted avg       0.99      0.99      0.99       114
```

In [38]:
```
jupyter nbconvert --to pdf BCDM_WTC\ project.ipynb
```

```
  Cell In[38], line 1
    jupyter nbconvert --to pdf BCDM_WTC\ project.ipynb
            ^
SyntaxError: invalid syntax
```

In [ ]: